

State Farm - Developer Guide

Date: July 10, 2026

Table of Contents

State Farm - Developer Guide	3
Quick Reference	3
Architecture Overview	3
Local Development Setup	6
Design System	7
Blocks Reference	8
Common Development Tasks	11
Environments	12
Git Workflow	13
Troubleshooting	13
Resources	14
Support Contacts	14

State Farm - Developer Guide

Quick Reference

Resource	URL
Code Repository	AdobeDrago/statefarm-eds (GitHub)
GitHub	https://github.com/AdobeDrago/statefarm-eds
Preview	https://main-statefarm-eds-adobedrago.aem.page/
Live	https://main-statefarm-eds-adobedrago.aem.live/
Content (DA)	https://content.da.live/adobedrago/statefarm-eds
Local Dev	http://localhost:3000

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+), no framework or build step
- CSS3 with Custom Properties (design tokens)
- Files served directly — no compilation or bundling required
- Content authored in Document Authoring (DA) at <https://content.da.live/adobedrago/statefarm-eds>

Project Structure

```
├── blocks/                # 11 custom blocks
│   ├── accordion-faq/
│   ├── cards-article/
│   ├── cards-checklist/
│   ├── cards-coverage/
│   ├── columns-agent-cta/
│   ├── columns-category/
│   ├── columns-promo/
│   ├── columns-subnav/
│   ├── footer/
│   ├── header/
│   └── hero-quote/
├── scripts/
│   ├── aem.js             # Core library (DO NOT MODIFY)
│   ├── scripts.js         # Custom page decoration & E-L-D phases
│   └── delayed.js         # Analytics and marketing tools (currently minimal)
├── styles/
│   ├── brand.css          # State Farm brand tokens (colors, typography)
│   ├── styles.css         # Critical global styles (imports brand.css)
│   ├── fonts.css          # Font declarations (lazy-loaded)
│   └── lazy-styles.css    # Non-critical global styles
└── head.html              # Custom <head> additions
```

```
└─ helix-query.yaml # Search/query configuration
└─ paths.json       # Path mapping configuration
```

Three-Phase Loading (E-L-D)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve a Lighthouse score of 100. Every file and script must be placed in the correct phase.

Phase	Purpose	What Loads	Performance Impact
Eager	Render above-the-fold content as fast as possible (LCP)	styles/styles.css, brand.css, first section blocks, scripts/scripts.js	Blocks LCP — keep minimal
Lazy	Load remaining page content after first paint	Remaining blocks, header, footer, fonts.css, lazy-styles.css	Runs after Eager — safe for non-critical UI
Delayed	Load non-essential third-party scripts	scripts/delayed.js (runs after 3-second timeout)	Never blocks rendering

Rules for developers: - Never add third-party scripts to scripts.js — they block LCP. Always use delayed.js. - Never load fonts eagerly — fonts.css is loaded lazily to avoid render-blocking. - Exception: on desktop (`window.innerWidth >= 900`) or when `sessionStorage.getItem('fonts-loaded')` is set, fonts are loaded eagerly as a performance optimization. - Blocks in the first section load eagerly; all others load lazily — this is automatic based on DOM position. - The header and footer load in the lazy phase via `loadHeader()` and `loadFooter()` from `aem.js`.

Key Files and How They Connect

File	Role	Connects To
scripts/aem.js	Core library — loadBlock, loadCSS, decorateBlock, sampleRUM, buildBlock, createOptimizedPicture, loadHeader, loadFooter. DO NOT MODIFY.	Imported by scripts.js and blocks
scripts/scripts.js	Entry point — orchestrates E-L-D phases. Contains buildAutoBlocks, buildWidgetAutoBlocks, decorateButtons, decorateMain, moveAttributes, moveInstrumentation.	Imports from aem.js; used by blocks
scripts/delayed.js	Loaded after a 3-second timeout — currently a	Called by scripts.js in delayed phase

File	Role	Connects To
	placeholder for future analytics/marketing tools.	
styles/brand.css	State Farm brand tokens: colors, typography, spacing, heading sizes. Single source of truth.	Imported by styles.css
styles/styles.css	Critical CSS + global styles. Loaded eagerly. Imports brand.css.	Referenced by all blocks and pages
styles/fonts.css	Font declarations (Source Sans 3 via Google Fonts). Loaded lazily.	Font families referenced in brand.css
styles/lazy-styles.css	Non-critical global styles. Loaded lazily.	Supplements styles.css
blocks/{name}/{name}.js	Block logic — exports default function <code>decorate(block)</code> . Auto-loaded when block class appears in DOM.	May import from <code>scripts.js</code> or <code>aem.js</code>
blocks/{name}/{name}.css	Block styles — auto-loaded alongside block JS.	Uses CSS custom properties from <code>brand.css</code>

Execution flow: page load → `scripts.js` → `loadEager()` (first section + eager blocks, fonts on desktop) → `loadLazy()` (remaining blocks, header, footer, `fonts.css`, `lazy-styles.css`) → `loadDelayed()` (3s timeout, loads `delayed.js`)

Custom scripts.js Functions

Function	Purpose	Exported
<code>buildWidgetAutoBlocks(main)</code>	Finds <code>/widgets/...</code> links and auto-converts them to widget blocks	No
<code>buildAutoBlocks(main)</code>	Finds <code>/fragments/...</code> links and auto-loads them as fragment blocks; calls <code>buildWidgetAutoBlocks</code>	No
<code>decorateButtons(main)</code>	Promotes <code><a></code> → <code>.button.primary</code> , <code><a></code> → <code>.button.secondary</code> , <code><a></code> → <code>.button.accent</code>	No
<code>moveAttributes(from, to, attributes)</code>	Transfers specified attributes from one element to another	Yes
<code>moveInstrumentation(from, to)</code>	Transfers Universal Editor instrumentation attributes (<code>data-aue-*</code> , <code>data-richtext-*</code>)	Yes

Function	Purpose	Exported
decorateMain(main)	Orchestrates full main decoration: icons, auto blocks, sections, blocks, buttons	Yes

Trusted Types policy: `scripts.js` registers a default Trusted Types policy that sanitizes `innerHTML` assignments — strips `<script>` tags from `createContextualFragment` sinks and removes `srcdoc` attributes from `iframes`. This is a security hardening measure required for modern browser environments.

Local Development Setup

Prerequisites

- Node.js (no specific version pinned in this project — LTS recommended)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
# Clone repository
git clone git@github.com:AdobeDrago/statefarm-eds.git
cd statefarm-eds

# Install dev dependencies (linting tools only — no build dependencies)
npm install

# Start local dev server
aem up --pages-url=https://main--statefarm-eds--adobedrago.aem.page
```

Local Server — Repoless/Multi-Site

This project uses a **repoless setup** — the `adobedrago` organization hosts 19 sites that share infrastructure. You must always pass `--pages-url` when starting the local server so the AEM CLI resolves content for the correct site:

```
aem up --pages-url=https://main--statefarm-eds--adobedrago.aem.page
```

Without `--pages-url`, the AEM CLI cannot resolve content for the `statefarm-eds` site and local preview will fail or show wrong content.

- Local URL: `http://localhost:3000`
- Auto-reloads on file changes
- Serves code locally, proxies content from preview

Linting

```
# Run all linting (JS + CSS)
npm run lint

# Auto-fix linting issues
npm run lint:fix
```

```
# Lint only JS
npm run lint:js

# Lint only CSS
npm run lint:css
```

The project uses ESLint (airbnb-base config) for JavaScript and Stylelint (standard config) for CSS. Husky runs lint automatically on commit.

Design System

Brand Tokens (`styles/brand.css`)

All design tokens are defined as CSS custom properties in `brand.css`. This file is the single source of truth for State Farm brand values.

Typography

```
--heading-font-family: "Source Sans 3", "Source Sans Pro", arial, helvetica, sans-serif;
--body-font-family:    "Source Sans 3", "Source Sans Pro", arial, helvetica, sans-serif;
```

Note on fonts: State Farm's proprietary typeface is MecherleSans (not publicly available). This project uses Source Sans 3 (Google Fonts) as a visually similar substitute. Fonts are declared in `fonts.css` and loaded lazily.

Heading Sizes

```
--heading-size-xxl: 39px; /* h1 */
--heading-size-xl:  33px; /* h2 */
--heading-size-l:   27px; /* h3 */
--heading-size-m:   22px; /* h4 */
--heading-size-s:   19px; /* h5 */
--heading-size-xs:  16px; /* h6 */
```

Body Font Sizes (defined in `styles.css`)

```
--body-font-size-m: 16px;
--body-font-size-s: 14px;
--body-font-size-xs: 12px;
```

Colors

```
--background-color: #fff;
--text-color:       #212121;
--link-color:       #d62311;
--link-hover-color: #0056b3;
--brand-red:        #d62311;
--brand-red-dark:   #a81b0d;
--section-light-bg: #f5f5f5;
--section-dark-bg:  #212121;
```

Spacing and Layout

```
--section-padding: 48px;  
--content-max-width: 1200px;  
--nav-height: 97px; /* 32px utility bar + 65px main nav */
```

Breakpoints

Name	Threshold	Usage
Mobile	0	Default (mobile-first) styles
Tablet	768px	@media (width >= 768px) — hero-quote typography boost
Desktop	900px	@media (width >= 900px) — main layout changes
Large	1200px (max-width)	Content boundary via --content-max-width

Section Styles

Apply these class names in page metadata or section metadata to style sections:

Class	Effect
light or grey	Light grey background (#f5f5f5)
dark	Dark background (#212121), white text and headings
accent	State Farm red background (#d62311), white text and headings
legal	Small 12px/16px text for legal disclaimer copy

Button Variants

Buttons are auto-decorated by decorateButtons() in scripts.js based on link formatting in DA:

Authored format	Class applied	Appearance
link	.button.primary	Red pill, white text
<i>link</i>	.button.secondary	White pill, red border and text
<i>link</i>	.button.accent	Red pill (high-impact CTA)

Blocks Reference

accordion-faq

Purpose: Expandable FAQ items using native HTML <details>/<summary> elements.

DOM transform: Each authored row (column 1 = question, column 2 = answer) becomes a <details class="accordion-faq-item"> with a <summary class="accordion-faq-item-label"> and a body <div class="accordion-faq-item-body">.

Dependencies: `moveInstrumentation` from `scripts.js` (for Universal Editor support).

Variants: No authored variants — all styling is consistent.

Notes: Uses native browser expand/collapse behavior. Universal Editor instrumentation is transferred from the authored row to the `<details>` element.

cards-article

Purpose: A grid of article cards, each with an optional image and body content (heading, text, links).

DOM transform: Block rows become `<ul>/<li>` elements. Columns containing only a `<picture>` get `.cards-article-card-image`; all other columns get `.cards-article-card-body`. Images are optimized to 750px width via `createOptimizedPicture`.

Dependencies: `createOptimizedPicture` from `aem.js`, `moveInstrumentation` from `scripts.js`.

Variants: No authored variants.

cards-checklist

Purpose: A grid of cards featuring checklist-style content (bullet points, numbered lists).

DOM transform: Identical pattern to `cards-article` — rows become `<ul>/<li>`, image columns get `.cards-checklist-card-image`, text columns get `.cards-checklist-card-body`. Images optimized to 750px.

Dependencies: `createOptimizedPicture` from `aem.js`, `moveInstrumentation` from `scripts.js`.

Variants: No authored variants. CSS styles `ol` markers in red (`--brand-red`) to match State Farm brand.

cards-coverage

Purpose: Insurance coverage option cards, typically showing product name, description, and a link.

DOM transform: Same `<ul>/<li>` pattern as other card blocks. `.cards-coverage-card-image` for image columns, `.cards-coverage-card-body` for text. Images optimized to 750px.

Dependencies: `createOptimizedPicture` from `aem.js`, `moveInstrumentation` from `scripts.js`.

Variants: No authored variants. Links are styled as red text buttons matching the brand.

columns-agent-cta

Purpose: A two-column promotional block for the “Find an Agent” call to action. Injects a live ZIP code form into the text column, replacing any authored “Find an agent” link.

DOM transform: - Image columns get `.columns-agent-cta-img-col` - Text column: any authored “Find an agent” link is removed; a `<form class="columns-agent-cta-form">` is injected after the last `<p>`, containing a ZIP code input and “Find an agent” submit button

Key behavior: The block programmatically replaces the static authored link with a functional form. The form’s submit action is handled by the browser (no JS submit handler — the button is `type submit`).

Variants: Column count class is auto-applied (e.g., `columns-agent-cta-2-cols`).

columns-category

Purpose: Multi-column layout for insurance category navigation links, each column optionally featuring an image.

DOM transform: Detects image-only columns and marks them with `.columns-category-img-col`. Column count class auto-applied.

Variants: Column count class (e.g., `columns-category-3-cols`).

columns-promo

Purpose: Promotional multi-column layout, typically used for marketing callouts with images and CTAs.

DOM transform: Same image-column detection pattern, marks image columns with `.columns-promo-img-col`. Column count class auto-applied.

Variants: Column count class (e.g., `columns-promo-2-cols`).

columns-subnav

Purpose: Sub-navigation row of links, displayed as a horizontal strip of anchor links within a section.

DOM transform: Image-column detection, marks with `.columns-subnav-img-col`. Column count class auto-applied.

Variants: Column count class (e.g., `columns-subnav-4-cols`). CSS hides non-link paragraphs and styles link paragraphs as inline navigation items.

header

Purpose: Full site header with a two-row layout: grey utility bar (Search, Help, Español) above a white main navigation (logo, primary nav links, login).

DOM source: Fetches `/nav.plain.html` (or a custom path from the block's authored link), which must contain four sections in order: 1. Brand section (State Farm logo) 2. Primary section (main nav links) 3. Utility section (Help, Español links) 4. Login section (Log In link)

DOM output:

```
.header
  .nav-utility-bar      <- grey strip: Search button + utility links
  .nav-search-pane      <- collapsible search panel (grid roll-down animation)
    .nav-search-form
      .nav-search-inner
        .nav-search-field <- floating label input
        button.nav-search-submit
        button.nav-search-close
  nav
    .nav-brand          <- State Farm logo
    button.nav-hamburger <- mobile only
    .nav-primary         <- main nav links with slide-in red underline on hover
    .nav-login           <- Log In link (red, bold)
```

Search pane behavior: The search pane uses CSS Grid `grid-template-rows: 0fr → 1fr` transition for a smooth roll-down animation. Toggle is controlled by `.is-open` class. Keyboard: Escape closes the pane.

Mobile behavior: Utility bar is hidden; hamburger button shows. On nav-open, primary nav and login stack vertically below the logo. Utility bar appears at the bottom of the open menu.

Variants: No authored variants. The search-open class is added to the block when the search pane is open.

Gotcha: The floating label in the search input requires `placeholder=" "` (a single space) in the HTML so the `:placeholder-shown` CSS pseudo-class works correctly.

hero-quote

Purpose: Full-width masthead block for insurance landing pages. Shows a heading, intro copy, product selection dropdown, and ZIP code input to start an insurance quote.

DOM source: Two-column authored block — first column is an image (small icon or full lifestyle hero), second column is heading + intro text.

DOM output: - Image column (`.hero-quote > div:first-child`): icon or full-bleed lifestyle photo - Text column (`.hero-quote > div:last-child`): heading, intro paragraph, injected quote form

Quote form fields: - Product select (Auto, Auto+Home, Auto+Renters, Motorcycle, Homeowners, Condo Owners, Renters, Small Business, Life, Boat, Medicare Supplement, Supplemental Health, Pet Insurance) - ZIP Code input (5 digits, `pattern="[0-9]{5}"`) - “Start a quote” submit button → POSTs to `/get-insurance-quote?productName=...&zipCode=...`

Key behavior: Any authored “Start a quote” link is automatically removed and replaced by the injected form. If no image is present, the `no-image` class is added to the block root.

Variants: | Variant | Behavior | `lifestyle` | Image column displays a large aspect-ratio hero photo (845:388 on mobile, `object-fit: cover` on desktop) instead of a small inline icon |

Layout: Mobile-first column stack. At 900px+, switches to a side-by-side row where the text column is pinned to the left content boundary and the image column bleeds to the right viewport edge.

footer

Purpose: Full-width site footer with a promotional banner, brand column, navigation link columns, legal links, and copyright.

DOM source: Fetches `/footer.plain.html` (or a custom path). Sections in order: 1. Banner (promotional strip) 2. Brand column (logo + tagline) 3. Link columns (repeating — all middle sections) 4. Legal links section 5. Copyright section

DOM output:

```
.footer-inner
  .footer-banner      <- full-width promotional strip
  .footer-content     <- brand column + link columns grid
    .footer-brand
    .footer-links (multiple)
  .footer-legal       <- legal links row
  .footer-copyright   <- copyright text
```

Common Development Tasks

Add a New Block

1. Create `blocks/{name}/` directory

2. Create `blocks/{name}/{name}.js` — must export default function `decorate(block)`
3. Create `blocks/{name}/{name}.css` — block styles
4. Author the block in DA and preview at `http://localhost:3000`
5. Import `moveInstrumentation` from `../../scripts/scripts.js` if Universal Editor support is needed
6. Push to a feature branch and check the feature branch preview

Modify Brand Colors or Tokens

1. Edit `styles/brand.css` — all design tokens live here
2. Do not scatter token definitions across individual block CSS files
3. Test across multiple pages, especially header and hero sections
4. Check both light and dark section variants

Add a New Section Style

1. Add the CSS rule in `styles/styles.css` under the `/* Sections */` block
2. Follow the pattern: `main .section.{name} { ... }`
3. If the section changes text color, also add heading color overrides

Add Analytics or a Marketing Tool

1. Add the integration to `scripts/delayed.js`
2. Never add third-party scripts to `scripts.js` — this blocks LCP
3. Test by checking the browser Network tab — the script should load after the 3-second delay
4. Confirm no render-critical logic is added to this file

Update the Header Navigation

1. Edit the `/nav` document in DA (at `https://content.da.live/adobedrago/statefarm-eds`)
2. The `nav` document must contain exactly 4 sections: brand, primary links, utility links, login
3. Preview the change at `http://localhost:3000`
4. The header block fetches `/nav.plain.html` — no code changes needed for content updates

Debug a Block

1. Check the browser console for errors during `decorate()` execution
2. Inspect the DOM — compare the authored structure (from `.plain.html`) against the expected output
3. Check that variant class names are spelled correctly (must match CSS selectors exactly)
4. Verify CSS specificity — block CSS uses `.block-name .element` selectors; global styles may conflict
5. Use the Network tab to confirm `{block}.css` and `{block}.js` loaded successfully (no 404s)

Environments

Environment	URL Pattern	Purpose
Local	<code>http://localhost:3000</code>	Development — requires <code>--pages-url</code> flag
Feature Branch	<code>https://{branch}-statefarm-eds-adobedrago.aem.page</code>	PR testing and review
Preview	<code>https://main-statefarm-eds-adobedrago.aem.page</code>	Staging / content preview

Environment	URL Pattern	Purpose
Live	https://main-statefarm-eds-adobedrago.aem.live	Production

Git Workflow

Branch Naming

- Features: `feature/description`
- Fixes: `fix/description`
- Keep names short — branch names appear in preview URLs and long names can cause issues

PR Requirements

1. Include the feature branch preview URL in the PR description
 2. Pass linting (`npm run lint`)
 3. Test on the feature branch preview — verify on mobile and desktop
 4. No console errors
 5. Check for CLS (layout shift) on block load
-

Troubleshooting

Block not loading?

1. Check the block folder name matches the CSS class name exactly (kebab-case)
2. Verify `{block}.js` exports `default function decorate(block)`
3. Check the browser Network tab for 404s on `{block}.js` and `{block}.css`
4. Confirm the block is authored in DA with the correct block name

Search pane not animating?

The search pane uses CSS Grid `grid-template-rows` transition. Ensure: - `.nav-search-form` has `min-height: 0` (required for `0fr` collapse) - The transition is on the `.nav-search-pane`, not the inner form - The `.is-open` class toggles `grid-template-rows: 0fr → 1fr`

Hero quote form not showing?

1. Verify the block has a second column (text cell) — if only one column exists, the function exits early
2. Check that any authored “Start a quote” link is detected by the case-insensitive regex
3. Confirm the DOM structure: `.hero-quote > div:last-child > div` must exist

Styles not applying?

1. Check CSS selector specificity — block CSS uses `.block-name .element` which may lose to global styles
2. Verify the CSS file loaded (Network tab — no 404s)
3. Check for CSS syntax errors (browser DevTools Styles panel shows parse errors)

Content not updating locally?

1. Content is proxied from the preview URL specified in `--pages-url`
2. Ensure the DA document was previewed (published to preview) before fetching locally
3. Clear browser cache or use incognito mode

Fonts not loading?

1. `fonts.css` is lazy-loaded — it appears after the first paint
 2. On desktop, fonts may be eagerly loaded if `sessionStorage.getItem('fonts-loaded')` is set
 3. To force reload fonts locally, clear `sessionStorage` in DevTools
-

Resources

Resource	URL
Edge Delivery Services Documentation	https://www.aem.live/docs/
Developer Tutorial	https://www.aem.live/developer/tutorial
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading Guide	https://www.aem.live/developer/keeping-it-100
Dev Best Practices	https://www.aem.live/docs/dev-collab-and-good-practices
Admin API Reference	https://www.aem.live/docs/admin.html
Document Authoring	https://da.live/

Support Contacts

For questions about this project, contact the AdobeDrago team via the GitHub repository: <https://github.com/AdobeDrago/statefarm-eds>